

PROJECTS

(stuck_at_ritesh)

DIGITAL DESIGN / RTL / FPGA

Basic (Must Do All 3)

1. Parameterized ALU

What it is:

An Arithmetic Logic Unit that performs operations such as:

- Addition
- Subtraction
- AND
- OR
- XOR
- Shifts
- Comparisons

Concepts learned:

- Combinational logic
- Multiplexing
- Arithmetic circuits
- Parameterization using Verilog generics/parameters

Industry relevance:

ALUs are present inside every processor.

Difficulty: 2/10

Resources:

- [Nandland Verilog Tutorials](#)
- [HDLBits Practice](#)

2. UART Transmitter & Receiver

What it is:

A communication protocol that sends data serially.

Example:

FPGA → PC

or

Microcontroller → FPGA

Concepts learned:

- FSM design
- Baud rate generation
- Timing accuracy
- Data framing

Industry relevance:

Very common peripheral protocol.

Difficulty: 4/10

Resources:

- UART Verilog Example GitHub Search
- [Nandland UART Tutorial](#)

3. Traffic Light / Elevator FSM

What it is:

Design a state machine controlling traffic lights or an elevator.

Concepts learned:

- Moore FSM
- Mealy FSM
- State transitions
- Control logic

Industry relevance:

Control units in processors and communication protocols are FSMs.

Difficulty: 3/10

Intermediate

4. SPI Master-Slave Protocol

What it is:

Implement Serial Peripheral Interface communication.

Signals:

- MOSI
- MISO
- SCLK
- CS

Concepts learned:

- Serial communication
- Clock synchronization
- Shift registers

Industry relevance:

Used in sensors, flash memories, ADCs, DACs.

Difficulty: 5/10

5. I2C Controller

What it is:

Implement I2C communication supporting:

- Start condition
- Stop condition
- ACK/NACK
- Read/Write

Concepts learned:

- Bidirectional buses
- Open-drain interfaces
- Protocol implementation

Industry relevance:

Used everywhere in embedded systems.

Difficulty: 6/10

6. FIFO with Assertions

What it is:

Memory buffer supporting:

- Write operations
- Read operations
- Full/empty detection

Add SystemVerilog assertions to verify correctness.

Concepts learned:

- Circular buffers
- Verification
- Assertions
- Coverage

Industry relevance:

Extremely common in ASIC designs.

Difficulty: 6/10

7. AXI-Lite Slave

What it is:

Implement an AXI-Lite peripheral.

Support:

- Register reads
- Register writes
- Handshaking

Concepts learned:

- Industry-standard bus protocols
- Handshake mechanisms
- Register design

Industry relevance:

Almost every SoC uses AXI.

Difficulty: 7/10

8. Cache Controller

What it is:

A memory hierarchy block managing:

- Cache hits
- Cache misses
- Main memory accesses

Concepts learned:

- Computer architecture
- Memory systems
- State machines

Industry relevance:

Core component of CPUs.

Difficulty: 8/10

9. SDRAM Controller

What it is:

Interface FPGA with external SDRAM.

Handle:

- Initialization
- Refresh cycles
- Read/Write transactions

Concepts learned:

- Timing constraints
- Memory protocols
- Controller design

Industry relevance:

Memory subsystem design.

Difficulty: 8/10

10. Calculator SoC

What it is:

Build a small processor-based system.

Components:

- ALU
- Registers
- FSM controller
- Display interface

Concepts learned:

- System integration
- Datapath design
- Processor basics

Industry relevance:

First step toward CPU design.

Difficulty: 7/10

Resources:

- [ZipCPU Blog](#)
- [OpenCores](#)

Advanced (Industry Level)

11. 5-Stage Pipelined RISC-V Processor

What it is:

Design a processor implementing:

Stages:

- 1. Fetch**
- 2. Decode**
- 3. Execute**
- 4. Memory**
- 5. Writeback**

Concepts learned:

- **Pipelining**
- **Hazards**
- **Forwarding**
- **Branch prediction basics**

Industry relevance:

Directly relevant to CPU companies.

Difficulty: 9/10

12. Superscalar RISC-V Core

What it is:

Execute multiple instructions simultaneously.

Concepts learned:

- **Instruction scheduling**
- **Out-of-order concepts**
- **Dependency handling**

Industry relevance:

Modern CPU architecture.

Difficulty: 10/10

13. CNN FPGA Accelerator

What it is:

Accelerate:

- Convolution
- Pooling
- Activation

using FPGA parallelism.

Concepts learned:

- DSP blocks
- Parallel architectures
- AI hardware

Industry relevance:

AI accelerators at NVIDIA, AMD, Intel.

Difficulty: 9/10

14. DDR3 Controller

What it is:

Implement complete DDR3 memory interface.

Concepts learned:

- High-speed interfaces
- Timing closure
- Calibration

Industry relevance:

Used in almost all SoCs.

Difficulty: 10/10

15. RISC-V + AXI + Cache Subsystem

What it is:

Build a mini SoC.

Contains:

- **Processor**
- **Cache**
- **AXI interconnect**
- **Memory**

Concepts learned:

- **SoC architecture**
- **System integration**
- **Protocols**

Industry relevance:

Closest to actual ASIC design work.

Difficulty: 10/10

16. Hardware Video Processing Pipeline

What it is:

Real-time image/video processing.

Examples:

- **Edge detection**
- **Grayscale conversion**
- **Blur filters**

Concepts learned:

- **Streaming architectures**
- **High-throughput design**
- **FPGA optimization**

Difficulty: 9/10

17. PCIe Endpoint Controller

What it is:

Implement a PCIe endpoint capable of communicating with a host system.

Concepts learned:

- **High-speed protocols**
- **Packet structures**
- **DMA concepts**

Industry relevance:

Networking, storage, accelerators.

Difficulty: 10/10

18. Ethernet MAC

What it is:

Implement Ethernet frame transmission and reception.

Concepts learned:

- **Networking hardware**
- **CRC generation**
- **Packet handling**

Industry relevance:

Networking ASICs and SoCs.

Difficulty: 8.5/10

19. FPGA Network Packet Processor

What it is:

Hardware engine that:

- Parses packets
- Filters packets
- Routes packets

Concepts learned:

- Networking architecture
- Pipelines
- Throughput optimization

Difficulty: 10/10

20. Tiny AI Accelerator ASIC Prototype

What it is:

Design a complete AI accelerator architecture.

Includes:

- MAC arrays
- SRAM blocks
- Controller
- Dataflow optimization

Concepts learned:

- Computer architecture
- Hardware acceleration
- ASIC thinking

Industry relevance:

Very close to work done at NVIDIA, AMD, Qualcomm, and AI hardware startups.

Difficulty: 10/10

For a B.Tech ENTCT student targeting TI/Qualcomm/Intel

The strongest progression is:

1. ALU
2. UART
3. SPI
4. FIFO + Assertions
5. AXI-Lite Slave
6. Cache Controller
7. Pipelined RISC-V

Completing these 7 projects properly (with testbenches, documentation, and GitHub repositories) already places you above the majority of fresh graduates applying for digital/VLSI roles.

Resources:

- [RISC-V Educational Core PicoRV32](#)
- [Rocket Chip Generator](#)
- [LiteX Framework](#)

ANALOG / MIXED SIGNAL

Basic (Must Do All 3)

1. Common Source Amplifier

What it is:

The MOSFET equivalent of a common-emitter BJT amplifier.

Input voltage is amplified at the output.

Concepts learned :

- MOSFET operation regions
- DC biasing
- Small signal analysis
- Gain calculation
- Input/output impedance

Typical specifications :

- Gain: 10–20 V/V
- Supply: 1.8V–5V

Industry relevance : Almost every analog block contains common-source stages.

Difficulty : 3/10

2. Differential Amplifier

What it is

Amplifies the difference between two input signals while rejecting common noise.

Concepts learned

- Differential gain
- Common-mode gain
- CMRR
- Tail current sources

Industry relevance: Input stage of almost every op-amp, ADC, and comparator.

Difficulty: 4/10

3. Current Mirror

What it is

Copies current from one branch to another.

Concepts learned

- Bias generation
- Current steering
- MOS matching
- Channel length modulation

Industry relevance

Found in virtually every analog IC.

Difficulty: 3/10

Resources:

- [LTspice Official](#)
- [All About Circuits Analog Tutorials](#)

Intermediate

(These are the projects that start resembling actual analog IP blocks used in chips.)

4. Two-Stage CMOS Op-Amp

What it is

A complete operational amplifier consisting of:

- **Differential input stage**
- **Gain stage**
- **Compensation network**

Concepts learned

- **Open-loop gain**
- **Phase margin**
- **Frequency response**
- **Stability**

Industry relevance: Most commonly asked analog interview project.

Difficulty : 7/10

5. Bandgap Reference

What it is

Generates a stable reference voltage that remains nearly constant despite temperature changes.

Typically around:

- **1.2V reference**

Concepts learned

- **PTAT currents**
- **CTAT voltages**
- **Temperature compensation**

Industry relevance : Used in ADCs, DACs, PMICs, regulators.

Difficulty : 8/10

6. Low Dropout Regulator (LDO)

What it is

Converts a varying input voltage into a stable output voltage.

Example:

- $5V \rightarrow 3.3V$

Concepts learned

- Feedback loops
- Pass transistors
- Load regulation
- Line regulation

Industry relevance

One of the most common analog blocks in industry.

Difficulty

8/10

7. PLL Building Blocks

What it is

Design individual PLL components:

- Phase Frequency Detector
- Charge Pump
- Loop Filter
- VCO

Concepts learned

- Frequency synthesis
- Clock generation
- Jitter

Industry relevance: Used in processors, DDR, USB, PCIe.

Difficulty: 8/10

8. SAR ADC Architecture

What it is

Design a Successive Approximation ADC.

Components:

- Comparator
- Capacitive DAC
- SAR Logic

Concepts learned

- Data conversion
- Quantization
- Sampling theory

Industry relevance

Extremely popular ADC architecture.

Difficulty

8/10

9. Charge Pump

What it is

Circuit that generates voltages higher or lower than the supply voltage.

Example:

- $1.8V \rightarrow 5V$

Concepts learned

- Capacitor switching
- Clocking circuits
- Efficiency analysis

Industry relevance: Used in flash memory and PLLs.

Difficulty: 7/10

10. Voltage Controlled Oscillator (VCO)

What it is

Oscillator whose frequency depends on input voltage.

Concepts learned

- Ring oscillators
- LC oscillators
- Phase noise
- Frequency tuning

Industry relevance: Core block inside PLLs.

Difficulty: 8/10

Resources: Analog IC Design Lectures by Behzad Razavi

Advanced

11. Complete 10-bit SAR ADC

What it is

A full ADC system including:

- Sample-and-hold
- Comparator
- Capacitive DAC
- SAR Logic

Concepts learned

- ENOB
- INL
- DNL
- Sampling noise

Industry relevance: Commercial ADC design.

Difficulty: 9/10

12. Fractional-N PLL

What it is

A highly accurate PLL capable of generating fractional frequencies.

Example:

- 2.456 GHz

instead of only integer multiples.

Concepts learned

- Delta-sigma modulation
- Fractional division
- Jitter analysis

Industry relevance

Wireless communication chips.

Difficulty: 10/10

13. High-Speed Comparator

What it is

Circuit that decides whether:

- $V_{in} > V_{ref}$
or
- $V_{in} < V_{ref}$

at very high speeds.

Concepts learned

- Regenerative latches
- Offset voltage
- Metastability

Industry relevance: ADCs and SerDes.

Difficulty: 9/10

14. Buck Converter IC

What it is

A switching regulator IC.

Example:

- $12V \rightarrow 1V$

with very high efficiency.

Concepts learned

- PWM control
- Inductor design
- Compensation networks
- Power electronics

Industry relevance: CPUs, GPUs, mobile devices.

Difficulty: 9/10

15. Low-Power Biomedical Front End

What it is

Amplifies tiny biological signals like:

- ECG
- EEG
- EMG

Concepts learned

- Noise analysis
- Instrumentation amplifiers
- Low-power design

Industry relevance: Medical electronics.

Difficulty: 9/10

16. Sigma-Delta ADC

What it is

A high-resolution ADC using oversampling.

Concepts learned

- Noise shaping
- Oversampling
- Decimation filters

Industry relevance: Audio and precision measurement systems.

Difficulty: 10/10

17. RF Low Noise Amplifier (LNA)

What it is

Amplifies weak RF signals received from an antenna.

Concepts learned

- **S-parameters**
- **Noise figure**
- **Impedance matching**

Industry relevance: WiFi, Bluetooth, 5G.

Difficulty: 10/10

18. CMOS Image Sensor Readout Chain

What it is

Design analog circuitry used to read pixel data from image sensors.

Blocks include:

- **Pixel amplifier**
- **Sample-and-hold**
- **ADC**

Concepts learned

- **Sensor interfaces**
- **Noise reduction**
- **High-speed analog design**

Industry relevance : Mobile camera chips.

Difficulty: 10/10

19. DDR PHY Analog Blocks

What it is

Design analog portions of DDR interfaces.

Blocks include:

- Delay locked loops
- Clock recovery
- IO drivers

Concepts learned

- High-speed signaling
- Timing calibration
- Signal integrity

Industry relevance: Memory interface design.

Difficulty : 10/10

20. SerDes Analog Front End

What it is

Design analog circuits for multi-gigabit communication links.

Used in:

- PCIe
- Ethernet
- USB

Concepts learned

- Equalization
- Clock recovery
- High-speed analog design

Industry relevance : Among the most challenging analog design areas.

Difficulty : 10/10

If you're an ENTC student aiming for Analog/Mixed-Signal roles, the strongest progression would be:

Foundation

1. Common Source Amplifier
2. Differential Amplifier
3. Current Mirror

Resume-Level Projects

4. Two-Stage CMOS Op-Amp
5. Bandgap Reference
6. LDO

Standout Project

7. Complete SAR ADC

A well-documented Op-Amp + Bandgap + LDO + SAR ADC portfolio is already strong enough to discuss in interviews for entry-level analog/mixed-signal roles because it covers amplification, biasing, feedback, power management, and data conversion—the core pillars of analog IC design.

Resources:

- Berkeley Analog Generator BAG Framework
- [SkyWater PDK](#)

EMBEDDED SYSTEMS

Basic Projects (Must Do)

These projects teach the peripherals, drivers, interrupts, and hardware-software interaction that every embedded engineer must know.

1. GPIO Driver Development

What it is: Control LEDs, switches, relays, and other digital devices through GPIO pins. (Example: Press button → LED turns ON)

Concepts learned

- Registers
- Bit manipulation
- Input/output configuration
- Hardware interfacing

Industry relevance Foundation of every embedded system.

Difficulty 2/10

2. UART Communication System

What it is

Send and receive serial data between:

- PC ↔ MCU
- MCU ↔ MCU

Example:

Type "HELLO" on terminal and display it on LCD.

Concepts learned

- UART protocol
- Interrupts
- Buffers
- Baud rate generation

Industry relevance Used everywhere for debugging and communication.

Difficulty 3/10

3. Temperature Monitoring System

What it is

Read data from a temperature sensor and display/log it.

Example:

LM35 → STM32 → LCD

Concepts learned

- ADC
- Sensor interfacing
- Calibration
- Data acquisition

Industry relevance Industrial monitoring and IoT systems.

Difficulty 4/10

Platforms:

- STM32
- ESP32
- MSP430

Resources:

- [STM32CubeIDE](#)
- [ESP-IDF Framework](#)

Intermediate Projects

(These are the projects commonly found on strong embedded resumes)

4. RTOS-Based Task Scheduler

What it is

Run multiple tasks simultaneously.

Example:

- Sensor Task
- Display Task
- UART Task

all running concurrently.

Concepts learned

- Task scheduling
- Context switching
- Semaphores
- Queues
- Mutexes

Industry relevance Most professional embedded systems use an RTOS.

Difficulty 7/10

5. CAN Bus Communication

What it is

Implement communication between multiple nodes using CAN.

Example:

- Vehicle ECU sends speed data
- Dashboard receives speed data

Concepts learned

- CAN frames
- Arbitration
- Error handling

Industry relevance Automotive industry standard.

Difficulty 7/10

6. Sensor Fusion IMU Project

What it is

Combine:

- Accelerometer
- Gyroscope
- Magnetometer

to determine orientation.

Concepts learned

- Kalman Filter
- Complementary Filter
- Sensor fusion

Industry relevance Drones, robotics, automotive systems.

Difficulty 8/10

7. Smart Energy Meter

What it is

Measure:

- Voltage
- Current
- Power
- Energy consumption

Concepts learned

- ADC processing
- Signal conditioning
- RMS calculations

Industry relevance Industrial and power electronics applications.

Difficulty 7/10

8. BLE Communication Stack

What it is

Build a Bluetooth Low Energy device.

Example:

Phone app controls LEDs on MCU.

Concepts learned

- BLE profiles
- Wireless communication
- Power optimization

Industry relevance Wearables and IoT devices.

Difficulty 8/10

9. Bootloader Development

What it is

Create firmware capable of updating itself.

Example:

Update firmware through UART or USB.

Concepts learned

- Flash memory management
- Firmware updates
- Memory mapping

Industry relevance Extremely valuable in industry.

Difficulty 8/10

10. Linux Device Driver

What it is

Write a Linux kernel driver for a custom device.

Example:

Character driver for GPIO control.

Concepts learned

- Kernel programming
- Device trees
- Interrupt handling

Industry relevance Linux dominates embedded systems.

Difficulty 8/10

Advanced Projects

(These projects are close to what embedded engineers work on professionally)

11. Automotive ECU Prototype

What it is

Develop an Engine Control Unit-like system.

Functions:

- Sensor acquisition
- Decision making
- Actuator control

Concepts learned

- Real-time systems
- Automotive architecture
- Diagnostics

Industry relevance Automotive embedded engineering.

Difficulty 9/10

12. ADAS Sensor Controller

What it is

Collect and process data from:

- Cameras
- Radar
- Ultrasonic sensors

Concepts learned

- Sensor interfaces
- Real-time processing
- Embedded AI basics

Industry relevance Autonomous driving systems.

Difficulty 9/10

13. Real-Time Drone Flight Controller

What it is

Control a drone's:

- Roll
- Pitch
- Yaw

using sensor feedback.

Concepts learned

- PID control
- Sensor fusion
- Real-time systems

Industry relevance One of the best embedded projects.

Difficulty 9/10

14. Custom Embedded Linux Board Bring-Up

What it is

Take a custom board from power-on to Linux boot.

Tasks:

- U-Boot
- Kernel configuration
- Device tree
- Drivers

Concepts learned

- Linux internals
- BSP development
- Hardware debugging

Industry relevance Highly respected skill.

Difficulty 10/10

15. Secure Boot + OTA Update Framework

What it is

Create a secure firmware update system.

Features:

- Firmware encryption
- Digital signatures
- OTA updates

Concepts learned

- Embedded security
- Cryptography
- Secure firmware management

Industry relevance Very relevant today.

Difficulty 9/10

16. AI Edge Device

What it is

Run machine learning models on embedded hardware.

Example:

- Face detection
- Object detection

using Raspberry Pi or Jetson.

Concepts learned

- TinyML
- Model optimization
- Hardware acceleration

Industry relevance Edge AI is rapidly growing.

Difficulty 8.5/10

17. Software Defined Radio (SDR)

What it is

Implement radio functions in software.

Applications:

- FM receiver
- Signal analyzer
- Communication systems

Concepts learned

- DSP
- RF fundamentals
- Real-time processing

Industry relevance Wireless communication companies.

Difficulty 9/10

18. Autonomous Robot Navigation

What it is

Build a robot that navigates without human intervention.

Uses:

- Sensors
- Mapping
- Path planning

Concepts learned

- Robotics
- Embedded control
- Sensor fusion

Industry relevance Robotics and automation.

Difficulty 9/10

19. Industrial Protocol Gateway

What it is

Convert between industrial protocols.

Example:

- Modbus ↔ CAN
- CAN ↔ Ethernet

Concepts learned

- Communication stacks
- Protocol translation
- Networking

Industry relevance Industrial automation.

Difficulty 8.5/10

20. Multi-Core RTOS System

What it is

Run multiple tasks across multiple processor cores.

Example:

- Core 1: Sensor processing
- Core 2: Communication
- Core 3: Control

Concepts learned

- SMP systems
- Inter-core communication
- Synchronization

Industry relevance Modern embedded processors.

Difficulty 10/10

If your goal is TI, Qualcomm, Intel, AMD, Nvidia

(The highest-value embedded progression would be)

Foundation

1. GPIO Driver
2. UART Communication
3. Sensor Monitoring

Resume-Level

4. RTOS Scheduler
5. CAN Bus
6. Bootloader
7. Linux Device Driver

Standout Project

8. Embedded Linux Board Bring-Up
or
9. Real-Time Drone Flight Controller

For semiconductor companies, **Bootloader Development**, **Linux Device Driver**, and especially **Embedded Linux Board Bring-Up** generally carry more interview value than typical IoT projects because they demonstrate low-level hardware understanding, debugging ability, memory management, interrupts, and operating system knowledge.

Resources:

- [FreeRTOS Official](#)
- [Embedded Artistry](#)
- [PX4 Autopilot GitHub](#)
- [Buildroot Embedded Linux](#)

Treat these projects as a reference and not a hardcore roadmap.

As I have always kept saying, the title of the project is not as important as the depth and clarity with which you do any project.

Best of luck in your VLSI Preparation Journey!!

Stay Hungry
ALWAYS